MPI and CUDA Parallel Sobel Edge

Jacob Hudnut	Oliver Centner	Matthew Taccetta	Elias Emons
jacob.hudnut@gmail.com	centno@rpi.edu	taccem@rpi.edu	emonse@rpi.edu
Programming	Programming	Writing	Writing

April 29, 2026

Abstract

This paper explores and analyzes a massively parallel algorithm for the Sobel boundary detection operation. By using Open MPI and NVIDIA CUDA C++, this program runs on RPI AiMOS using its Power9 system architecture to read a color image and output the black and white image boundary. The implementation leverages distributed memory parallelism through MPI alongside GPU acceleration via CUDA to efficiently process large image datasets. Workloads are partitioned across multiple ranks and GPUs, enabling concurrent execution of kernel convolution operations central to the Sobel operation. The project evaluates performance in terms of strong scalability, weak scalability, and runtime, highlighting the advantages of hybrid parallel programming models. Additionally, it investigates communication overhead, memory transfer costs between CPU and GPU, and optimization strategies such as overlapping computation with data movement. The results demonstrate significant improvements in core algorithm time compared to sequential approaches, showcasing the effectiveness of combining MPI and CUDA for high-performance image processing applications.

Keywords

Gradient, Operator, Sobel Edge Detection, Parallel Processing, Image Edge detection, Hardware, Sobel Operator, Pixel Data, Algorithm, Digital Image Processing

Introduction

A crucial aspect of the field of image processing is the excess of redundant and repetitive data contained within any given image file (Vincent and Folorunso, 2009). Over the years of the field's existence, researchers have come to find that the most important data of an image tends to lie in it's edges (Vincent and Folorunso, 2009). As a result, edge detection operations perform a crucial role in the entire field of image processing. One such operator is the Sobel operator, designed by Irwin Sobel and Gary M. Feldman, the Sobel operator makes use of a pair of 3 by 3 kernels to produce an approximated gray-scale image gradient, from which edges can be found and defined (Sanduja and Patial, 2012). In addition to the Sobel Operator, this project makes use of the Message Passing Interface, or MPI. MPI enables message passing programming, which serves as an important and fundamental building block in the field of parallel programming. It's core principles revolve around send and receive functions, functions which enable parallelization via different processes (called ranks), being able to communicate between each other and synchronize when need be. MPI has been used alongside Sobel operator algorithms before (Abdul Khalid et al., 2011) and will be a major aspect of allowing this project to run in parallel. On top of MPI, this project also incorporates the Compute Unified Device Architecture, or CUDA. CUDA is a platform developed by the Nvidia corporation as a way to allow a program run on a "Host", defined as a system's main CPU and memory, to run parallel code on a system "De-

vice", defined as a systems Graphics Processing Unit or GPU. With CUDA, memory is copied from the Host memory to Device memory before the device loads it's own program, and executes it. After the device code has been executed, the device memory is then copied back to the host memory. Both MPI and CUDA support multiple languages, but for this specific project, they were implemented using the C programming language. Finally, all programs described by this paper were run on the Artificial Intelligence Multiprocessing Optimized System, or AiMOS, supercomputer at Rensselaer Polytechnic Institute, or RPI. AiMOS is a supercomputer introduced in 2019 that operates off of the IBM POWER-9 architecture. Students at RPI who require the AiMOS supercomputer for academic purposes may be given access, after which they can remotely access AiMOS and queue for compute nodes to run code designed for the parallel architecture of AiMOS. AiMOS supports code using MPI and CUDA.

Review of Related Works

One article, "Design of an image edge detection filter using the sobel operator", from Kanopoulos et al., 1988, provides a relative groundwork for the computation of a Sobel operator driven algorithm. In it, while there are mentions of parallel image processors (Kanopoulos et al., 1988, p. 358), the end goal of the report is to describe a non parallel, application specific chip designed entirely for processing an image via the Sobel operator. This, while helpful as a baseline, is not entirely useful in a modern setting, where parallel implementations on non-purpose built systems and chips are much more practical. Another, more modern article, is "Analysis of parallel multicore performance on sobel edge detector", from Abdul Khalid et al., 2011. Here, the implementation of a Sobel operator driven algorithm is given that takes advantage of MPI and a multi-core processor to speed up the operation of a Sobel operator driven algorithm (Abdul Khalid et al., 2011, pg. 316). This implementation, while a higher level of parallelism than the previous article described, still falls short of the parallel implementation described by this project, as it lacks the use a CUDA enabled device to allow for parallel implementation on a Graphics Processor.

Parallel Algorithm

This projects parallel algorithm takes the input image binary, calculates the total number of pixels within the image, and turns it to grayscale. After the grayscale transformation is complete, the total pixels are divided among the processes which each use the Sobel operator for boundary detection.

Sobel Edge Detection Operator

The Sobel Operator uses two 3x3 convolution kernels G_X and G_Y such that

$$G_X = \begin{array}{|c|c|c|}\hline -1 & 0 & 1 \\ \hline -2 & 0 & 2 \\ \hline -1 & 0 & 1 \\ \hline \end{array}, \quad G_Y = \begin{array}{|c|c|c|}\hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array}.$$

G_X is the kernel/mask that detects horizontal lines while G_Y detects vertical lines. By using the convolution of both of these kernels over the grayscale image, two different values are calculated for each pixel. One that represents the gradient component (or change in brightness) with respect to the horizontal (Gx). The other represents the gradient component with respect to the vertical (Gy). Using Pythagorean Theorem it is possible to combine the two gradient components to find the absolute magnitude of the gradient ($|G|$) at each pixel:

Figure 1: RGB Image Before Edge Detection

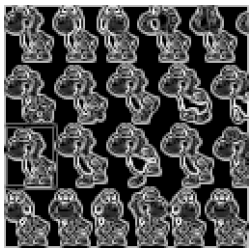

Figure 2: Grayscale Image after Edge Detection

The Sobel Edge Operator also results in an averaging effect over the image, so artifacts due to unwanted noise in the image are cleaned up by some amount.

Parallel Implementation

The objective of this implementation is for Edge Detection using the Sobel operator for digital image processing and implementation OpenMPI and CUDA C. Firstly, a RGB image in byte format (.data) is inputted and converted to gray scale. This image is read into the base MPI rank, where pixel objects are created to track each pixel's neighbors, and the data is scattered to all MPI ranks. Next, each process uses CUDA to perform the Sobel Edge Detection using the Sobel operator. The Sobel operator is a first order edge detection operator, which computes an approximation of the gradient of the image intensity function. At each pixel in the image, the result of the Sobel operator is the corresponding norm of this gradient vector. The Sobel operator only considers the two orientations which are horizontal and vertical convolution kernels. The operator uses the two kernels which are convolved with the original image to calculate approximations of the gradient. Next, scan the text file with the window and find out the values of center pixel, top, bottom, right, left, bottom-right, bottom-left, top-right and top-left pixel for the whole binary image row wise and column wise, this completes the horizontal and vertical scanning. Finally, the updated pixels are written to an output file using MPI Parallel I/O. This process flow is shown in figure 3.

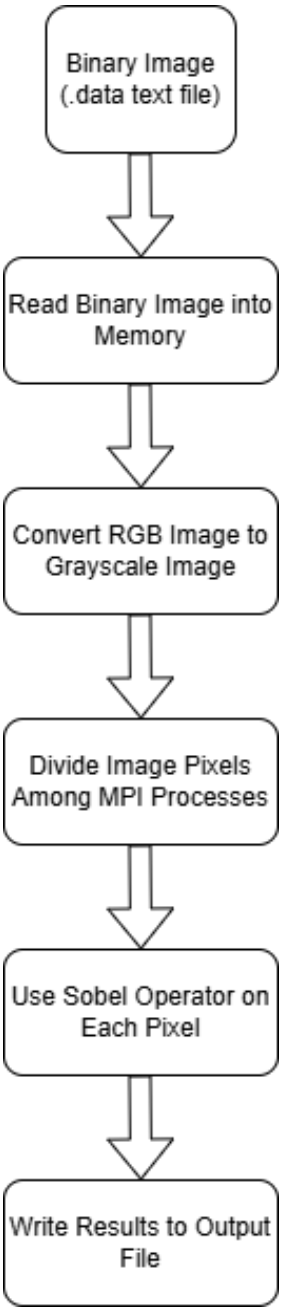

Figure 3: Program Process flow

Pseudocode

Algorithm 1 Parallel Sobel Edge Operator

Require: Input image I of size $W \times H$
Ensure: Output edge image E

Define Sobel kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Allocate output image E of size $W \times H$
for all pixels (x, y) in parallel **do**
  if $x = 0$ or $x = W - 1$ or $y = 0$ or $y = H - 1$ **then**
    $E(x, y) \leftarrow 0$
  else
    $sum_x \leftarrow 0$
    $sum_y \leftarrow 0$
    for $i \leftarrow -1$ to 1 **do**
      for $j \leftarrow -1$ to 1 **do**
        $pixel \leftarrow I(x + j, y + i)$
        $sum_x \leftarrow sum_x + pixel \cdot G_x(i+1, j+1)$
        $sum_y \leftarrow sum_y + pixel \cdot G_y(i+1, j+1)$
      end for
    end for
    $magnitude \leftarrow \sqrt{sum_x^2 + sum_y^2}$
    $E(x, y) \leftarrow \min(255, magnitude)$
  end if
end for
return E

Performance Results

Below is the CUDA Runtime data and Total Runtime data for both Weak Scaling and Strong Scaling. All results are represented in this section as per the process flow above. Please note that all runtimes were measured using the machine code provided by Professor Chris Carothers on Rensselaer Polytechnic Institute's AiMOS supercomputer and are reported in seconds. Additionally, for strong scaling, all of the tests use the same 18432x18432 pixel size image. For weak scaling the image size increases from 11585x11585 pixels to 46341x46341 pixels and are listed under dimension.

Table 1: Strong Scaling Runtimes with Varying Node and MPI rank quantities

Rank	Node	CUDA Time (s)	Total Time (s)
1	1	5.35	8.73
2	1	2.88	5.80
4	1	1.43	3.79
8	2	0.75	4.01
16	3	0.47	4.30
32	6	0.29	3.91

Table 2: Weak Scaling Runtimes with Varying Node and MPI rank quantities

Rank	Node	Dimension	CUDA (s)	Total (s)
1	1	11585	2.25	3.67
2	1	16384	2.36	4.81
4	1	23170	1.92	7.12
8	2	32768	1.90	11.50
16	3	46341	2.17	23.14

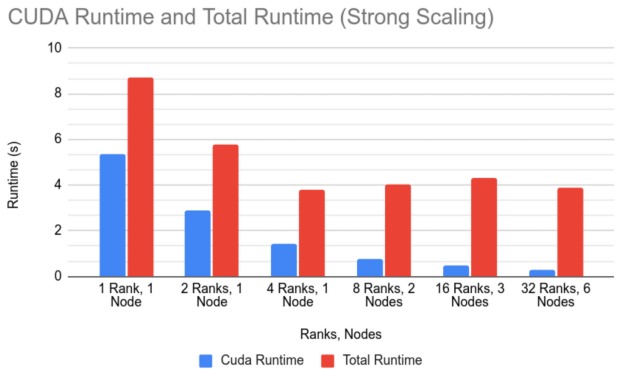

Figure 4: Strong Scaling Graph

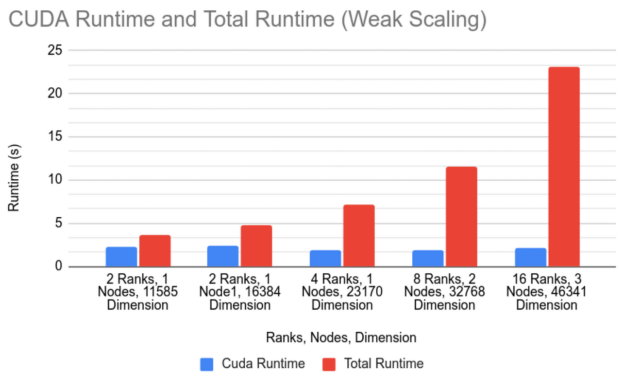

Figure 5: Weak Scaling Graph

Analysis of Performance Results

Firstly, as one can see from figures 4 and 5 as well as tables 1 and 2, the runtime is split into two measurements. One for CUDA runtime and another for Total runtime. On the one hand, CUDA runtime represents the time the program spends performing the Sobel Operation on each pixel in parallel. On the other hand, the Total runtime is the entire runtime of the program. This means that the difference between the Total runtime and CUDA runtime is equal to the overhead from MPI I/O as well as AiMOS and the rest of the C and CUDA code.

Strong Scaling Discussion

With respect to the CUDA runtime, the strong scaling performance is similar to the expected results. As the number of ranks and nodes increase CUDA runtime decreases by approximately 40% from the previous run. However, the Total runtime does not follow the same trend. Once the program reaches 4 MPI ranks, the Total runtime becomes nearly constant, suggesting that the overhead from MPI I/O and other sources become a bottleneck for performance.

Weak Scaling Discussion

Similarly to Strong Scaling, the CUDA runtime of the weak scaling performance test matches the expectation. Note that while dimension 11585 to dimension 16384 does not appear to be twice the problem size at first glance the problem is being properly scaled. Since these are square images, dimension is the side length of a square and thus the square root of the corresponding image area. This means that the increase in dimension results in the area of the image being doubled rather than side length. As is shown in figure 5, the CUDA runtime stays approximately the same across runs as problem size and parallelization both increase linearly. However, the Total runtime does not match the expected results once again. As the image size and Rank increase the Total runtime increases dramatically. Once again this suggests that the overheads for the program have a larger impact on Total runtime than the Sobel Edge Detection implementation.

Conclusion and Future Work

Overall, a parallel implementation of an algorithm that incorporates the Sobel operator can be said to have runtime improvements over a non parallel implementation of the algorithm. By using multiple processes via MPI, alongside parallelization through CUDA device code, a noticeable decrease in runtime can be observed. It should be noted, however, that the majority of the performance gains appear to only manifest in the CUDA runtime, as the total runtime outside of CUDA appears to stay relatively the same throughout strong scaling tests, and increase significantly across weak scaling tests. This suggests that the overhead of this project's image processing algorithm prove to be the more significant bottleneck. It is possible that a majority of this overhead comes from the MPI/IO implementation not providing significant benefit. Therefore the CUDA side of the parallelization drives the performance gain seen in the data. As for the implementation itself, putting the Sobel operation into a parallel format allowed for easy implementation of both MPI/IO and CUDA operations into the code. Therefore, it can be said that the cost of implementing a parallel form of an edge detection algorithm using the Sobel Operator does not offset the performance gains.

Future Work

As for future work, there are multiple implementations of the Sobel operator where a parallel approach could similarly be applied. For example, the Sobel operator is often used in algorithms designed to approximate the angle of an object seen in an image. This is often seen in manufacturing, where the pitch of threads on a screw can be measured via digital imaging (Jing and Du, 2020). It would be possible to take the parallel approach as described in this paper and apply it to Sobel implementations of angle approximation algorithms. In addition to implementations of the Sobel operator that were not covered by this paper, there is also further additions that could be made to the algorithm that was designed for this project. In the program made for this project, zero padding was used to account for calculations of pixels on the corner of images. However, there are other padding implementations that are possible beyond zero padding. For example, border pixels could be ignored and left out in the output, or, the image could be more properly extended by estimation via derivatives of the gray scale image.

References

Abdul Khalid, N., Ahmad, S., Noor, N., Ahmad Fadzil, A., & Taib, M. (2011). Analysis of parallel multicore performance on sobel edge detector. *WSEAS International Conference on Computers*, *15*, 313–318.

Chaple, G. N., Daruwala, R. D., & Gofane, M. S. (2015). Comparisions of robert, prewitt, sobel operator based edge detection methods for real time uses on fpga. *2015 International Conference on Technologies for Sustainable Development (ICTSD)*, 1–4. https://doi.org/10.1109/ICTSD.2015.7095920

Fredj, H. B., Ltaif, M., Ammar, A., & Souani, C. (2017). Parallel implementation of sobel filter using cuda. *2017 International Conference on Control, Automation and Diagnosis (ICCAD)*, 209–212. https://doi.org/10.1109/CADIAG.2017.8075658

Jain, A., Namdev, A., & Chawla, M. (2016). Parallel edge detection by sobel algorithm using cuda c. *2016 IEEE Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, 1–6. https://doi.org/10.1109/SCEECS.2016.7509360

Jing, M., & Du, Y. (2020). Flank angle measurement based on improved sobel operator. *Manufacturing Letters*, *25*, 44–49.

Kanopoulos, N., Vasanthavada, N., & Baker, R. (1988). Design of an image edge detection filter using the sobel operator. *IEEE Journal of Solid-State Circuits*, *23*(2), 358–367. https://doi.org/10.1109/4.996

Priyanka, V., Rama, Y. S., Sravani, K., & Kavya, B. (2024). Implementation of sobel edge detection with image processing on fpga. *2024 2nd World Conference on Communication & Computing (WCONF)*, 1–5. https://doi.org/10.1109/WCONF61366.2024.10692301

Sanduja, V., & Patial, R. (2012). Sobel edge detection using parallel architecture based on fpga. *International Journal of Applied Information Systems*, *3*(4).

Vincent, R., & Folorunso, O. (2009). A descriptive algorithm for sobel image edge detection. *Proceedings of Informing Science & IT Education Conference*, 97–107.

Yasri, I., Hamid, N., & Yap, V. (2008). Performance analysis of fpga based sobel edge detection operator. *2008 International Conference on Electronic Design*, 1–4. https://doi.org/10.1109/ICED.2008.4786751